

3D Tic-Tac-Toe Final Report

Abstract

This paper will focus on developing a well-suited automated version of a 3D Tic-Tac-Toe game. It will focus on the implementation of game trees and the MiniMax algorithm enhanced with alpha-beta pruning and Monte Carlo Tree Search (MCTS). The key challenge is to effectively model the numerous possible states of a game tree. Each node in this tree represents a potential state of the game, arranged by turns, leading from an initial state (the empty grid) through various player moves, to terminal states (win, lose, or draw). The ultimate goal is to create two robust AI players capable of making quick, strategic decisions during gameplay, enhancing both the complexity and entertainment value of the game. This study emphasizes the importance of game tree modeling and showcases how algorithmic efficiency plays a critical role in developing AI for complex, multidimensional games.

1 Introduction

Artificial intelligence (AI) plays a vital role in game development, offering techniques to design intelligent and competitive opponents across different game settings. The development of a 3D Tic-Tac-Toe game introduces a set of unique challenges compared to the traditional 2D version. This extension of Tic-Tac-Toe to a 3D grid, specifically a 4x4x4 layout, amplifies the complexity of the state space, making advanced AI techniques essential for effective decision-making. In total, there are 64 cells and an exponentially larger number of potential winning combinations, resulting in a game tree of significantly increased size and complexity. The interesting challenge lies in efficiently modeling the various possible states of this game tree. Each node represents a potential state of the game, arranged by turns, leading from an initial state (the empty grid) through various player moves to terminal states (win, lose, or draw). The ultimate objective is to create a robust AI capable of making optimal moves in real time, providing an engaging and strategically challenging experience for human players.

This study demonstrates the significance of game tree modeling and highlights the role of algorithmic efficiency in AI development for complex, multidimensional games. Designing an AI that can navigate this expanded game space efficiently and intelligently requires advanced algorithmic strategies to handle the exponentially larger state space. The Minimax algorithm, combined with alpha-beta pruning, is a central approach in this project. As well as modeling the game using advanced tree structures and algorithms, such as Monte Carlo Tree Search. These algorithms work together to reduce the computational complexity involved in evaluating all possible outcomes, ensuring that the AI operates within a reasonable time frame. By leveraging these techniques, researchers and developers can optimize the AI's ability to make real-time decisions, balancing computational demands with the need for strategic depth. Furthermore, Monte Carlo Tree Search (MCTS) offers an additional layer of optimization by simulating multiple potential outcomes to inform the AI decision-making process. This project draws on research about decision-making algorithms, including alpha-beta pruning and MCTS, decision-making algorithms, and multi-agent systems, examining how they can be applied to create an AI for 3D Tic-Tac-Toe.

The challenges posed by a 4x4x4 grid extend beyond simple scaling, as the three-dimensional layout introduces new layers of complexity in identifying winning combinations and prioritizing moves. Developing an AI agent for 3D Tic-Tac-Toe requires leveraging various principles of artificial intelligence and game theory to address these challenges. The sequence and combination of heuristic functions play a crucial role in navigating the expanded decision space, often leading to measurable performance improvements. Heuristics guide AI in effectively evaluating board positions, enabling it to prune irrelevant branches of the game tree and focus on the most promising strategies. Given the complexity of the 3D grid, game agents must utilize advanced tree search algorithms, such as Minimax, alpha-beta pruning, or Monte Carlo Tree Search, to optimize efficiency. This combination of strategies enables the AI to evaluate a large number of state-of-game while maintaining strategic accuracy.

The development of this project highlights the intricate balance between algorithmic sophistication and computational efficiency. Implementing a 3D Tic-Tac-Toe game agent requires a deep understanding of AI techniques and game theory, focusing on strategies that improve decision making, improve performance, and adapt to the unique challenges posed by three-dimensional gameplay. By addressing these challenges, the resulting AI not only meets the demands of a complex gaming environment but also sets the stage for advancements in AI applications in multidimensional problem-solving scenarios.

2 Tree Search Algorithms and Optimization

Tree search algorithms are foundational to artificial intelligence (AI) in games, enabling systematic exploration of potential game states to determine optimal strategies. These algorithms provide a framework for modeling the decision-making process of an AI agent, where each node in the search tree represents a possible configuration of the game at a specific point in time. In the case of 3D Tic-Tac-Toe, the game space extends significantly beyond the traditional 2D version, requiring advanced methods to efficiently navigate the decision tree.

The MiniMax algorithm assumes both players will make the best moves possible. It looks at all possible moves to figure out the best one but can take a long time because there are so many options in 3D Tic-Tac-Toe. MiniMax explores all possible outcomes to determine the optimal move, but its computational requirements grow exponentially with the depth of the game tree. For a 4x4x4 Tic-Tac-Toe game, this depth is considerable, given the exponential growth in game states.

To mitigate this challenge, alpha-beta pruning is employed. Alpha-beta pruning enhances the efficiency of MiniMax by eliminating branches of the game tree that will not influence the final decision. By skipping irrelevant calculations, alpha-beta pruning reduces the effective branching factor, allowing the algorithm to explore deeper levels of the game tree within the same computational constraints. Allis' work on heuristic evaluation functions further demonstrates how computational costs can be minimized while maintaining decision quality [2].

Recent advancements have extended alpha-beta pruning for more dynamic and complex scenarios. Buro's work, for example, generalizes alpha-beta pruning to accommodate simultaneous moves, a feature relevant for games like 3D Tic-Tac-Toe where player actions may be influenced by overlapping strategies or adaptive gameplay conditions [4]. Applications of alpha-beta pruning in games such as checkers validate its utility, as demonstrated by Jofanda and Yasin, who structured their decision tree to prioritize optimal moves while minimizing computational overhead [8].

Depth-First Iterative Deepening (DFID) offers another optimization strategy by combining the memory efficiency of depth-first search with the completeness of breadth-first search. This method is particularly suited for large search spaces like those in 3D Tic-Tac-Toe, where resource management is critical. Korf and Hsu's implementation of DFID highlights its effectiveness in managing large-scale problems, making it an invaluable tool for AI design in complex games [10].

3 Monte Carlo Tree Search and Evaluation

Monte Carlo Tree Search (MCTS) has emerged as a powerful method for navigating large and complex game trees, particularly in scenarios where predefined evaluation functions are unavailable. Monte Carlo Tree Search (MCTS) offers a probabilistic alternative to deterministic algorithms like Minimax. By simulating multiple potential game outcomes, MCTS dynamically evaluates states and adapts strategies based on statistical insights. MCTS leverages stochastic simulations to evaluate game states, dynamically adapting to the evolving nature of gameplay. Its adaptability makes it an attractive choice for 3D Tic-Tac-Toe, where the branching factor is significantly larger than traditional games.

Chaslot's studies demonstrate the versatility of MCTS in games like poker and Scrabble, which share characteristics of high-dimensional decision spaces with 3D Tic-Tac-Toe [6]. Unlike deterministic approaches like MiniMax, MCTS uses random rollouts to simulate future game states, making it effective in handling uncertainty. Variants like Nested Monte Carlo Search (NMCS), introduced by Poulding and Feldt, deepen

decision-making by iteratively simulating nested game states, which can improve strategic planning in multi-dimensional games [14].

Comparative studies, such as Kato’s work on the game of Amazons, highlight the advantages of integrating MCTS with heuristic evaluations. While MCTS performs well without heuristics, its performance improves significantly when combined with tailored evaluation functions [9]. In the context of 3D Tic-Tac-Toe, this combination allows the AI to balance exploration of new strategies with exploitation of proven ones.

Winands explored the application of MCTS in Lines of Action, emphasizing how it dynamically evaluates states and adapts strategies to complex rule sets [15]. This adaptability is crucial for 3D Tic-Tac-Toe, where the AI must account for the expanded game board and intricate win conditions.

4 Heuristic Evaluation and Decision-Making

Heuristic evaluation functions play a pivotal role in enhancing the efficiency of AI decision-making. These functions assign numerical scores to intermediate game states, guiding the AI toward favorable positions while avoiding disadvantageous ones. For 3D Tic-Tac-Toe, heuristics must account for the increased number of winning lines and prioritize moves that maximize strategic advantage.

Allis’ foundational work on heuristic functions underscores their importance in reducing computational demands while maintaining strategic depth [2]. Brown expanded on this concept by developing heuristics tailored to three-dimensional games, emphasizing the balance between offensive and defensive strategies [3]. For example, heuristics may assign higher weights to moves that block an opponent’s potential win or extend the AI’s own winning sequence.

Lisý proposed alternative selection functions for MCTS, designed to improve search efficiency in scenarios with large branching factors, a concept particularly relevant to 3D Tic-Tac-Toe [11]. These heuristics enhance the AI’s ability to evaluate game states in real time, ensuring that decisions are both computationally feasible and strategically sound.

5 Challenges in 3D AI Design

The expansion from a 2D to a 3D grid introduces exponential growth in the number of game states, presenting significant challenges in state space management. This is a critical issue in designing AI for 3D Tic-Tac-Toe, requiring robust algorithms to efficiently traverse the decision tree. Abbott et al. analyzed the computational complexity of two-player games, highlighting the importance of advanced algorithms like Minimax with alpha-beta pruning [1]. Lu and Li emphasized the potential of adaptive learning to dynamically manage large search spaces [12].

Cazenave proposed Sequential Halving, an enhancement to MCTS that prioritizes promising branches, improving computational efficiency in large search spaces [5]. Debugging and refining the AI’s decision-making processes is another key challenge. Patashnik suggested adaptive pruning strategies to handle diverse scenarios, such as stalemates and edge cases. These techniques ensure consistent performance even in complex game situations.

Churchill and Buro provided a framework for managing large search spaces using hierarchical portfolio search [7]. While developed for real-time strategy games, this approach can inform the design of AI for 3D Tic-Tac-Toe, where managing computational resources is paramount.

6 Problem Approaches

The approach to solving the problem of creating an effective AI for 3D Tic-Tac-Toe involves developing a comprehensive game tree structure and implementing advanced algorithms to navigate the decision space efficiently. The primary goal is to represent all possible states of the game and provide the AI with the tools necessary to make optimal decisions in real-time. The game tree will capture all potential configurations of the board, starting from the initial empty state, and progressively branching into all possible moves made by the players.

To navigate this tree, the MiniMax Algorithm will serve as the foundation for evaluating potential outcomes of each move for both the maximizing (AI) and minimizing (human) players. To address the computational complexity associated with the large state space of 3D Tic-Tac-Toe, the algorithm will be enhanced with Alpha-Beta Pruning. This optimization will significantly reduce the number of nodes the algorithm needs to evaluate, making the decision-making process more efficient. By combining these techniques, the AI will be capable of consistently selecting the best possible moves based on the current game state.

6.1 Implementation

The 3D Tic Tac Toe game was implemented using Python due to its flexibility and the availability of libraries that streamline AI algorithm development. This project focused on creating AI strategies to play the game effectively, emphasizing AI-versus-AI interactions to evaluate decision-making and strategic planning. The state of the game is represented as a 3D grid, and different AI agents use various algorithms to determine the best moves in a competitive environment.

6.2 Game Representation

The 3D Tic Tac Toe game is modeled as a 4x4x4 grid with 64 cells. Each cell can hold one of three states: empty, occupied by AI Player 1, or occupied by AI Player 2. This representation allows both AI agents to evaluate the board state and simulate potential moves. Unlike the traditional 3x3 grid, this three-dimensional representation introduces additional layers of complexity due to the higher number of winning combinations.

Winning lines in 3D Tic-Tac-Toe can occur:

- Along rows, columns, and diagonals within any of the four planes (XY, XZ, and YZ).
- Vertically through the grid along the Z-axis.
- Along 3D diagonals, spanning all three dimensions.

The total number of potential winning lines increases significantly compared to the 2D case, which must be accounted for in the AI's evaluation function. Efficient representation of these winning lines is critical for both move validation and heuristic evaluation.

The game tree is a hierarchical data structure used to model the decision-making process. Each node in the tree represents a specific board configuration:

- The root node corresponds to the initial empty state.
- Edges between nodes represent valid moves made by either player.
- Child nodes correspond to the resulting game states after a move is played.
- Leaf nodes represent terminal states, where the game ends in a win, loss, or draw.

Given the high branching factor of a 4x4x4 grid, the game tree expands rapidly with each move. A full traversal of the tree is computationally infeasible, making optimization strategies like Alpha-Beta Pruning essential for reducing unnecessary exploration.

7 Algorithmic Approach

Three AI strategies were implemented and evaluated in head-to-head matches:

Minimax with Alpha-Beta Pruning: This algorithm explores all possible moves in the game tree but uses alpha-beta pruning to skip unnecessary branches. This significantly reduces the time it takes to calculate the best move while still ensuring an optimal decision. The algorithm assigns scores to the outcomes: +1 for a win, -1 for a loss, and 0 for a draw.

Minimax Without Alpha-Beta Pruning: To compare the efficiency of pruning, I also implemented a basic version of minimax that doesn't include optimizations. This version calculates the best move by evaluating all game states exhaustively, but it is slower and less practical for larger games.

Monte Carlo Simulation: This algorithm evaluates potential moves by simulating random games from the current state. Each move is scored based on the percentage of games it leads to a win. The move with the highest score is chosen. This approach doesn't guarantee the best move every time, but it's useful for exploring a large number of possibilities quickly.

7.1 MiniMax Algorithm

A MiniMax algorithm is a recursive algorithm for choosing the next move in an n-player game. The MiniMax algorithm is central to the AI's decision-making process. It recursively evaluates all possible game outcomes from the current state, assuming that both players act rationally:

- The maximizing player (AI) aims to maximize the score by selecting moves that improve its chances of winning.
- The minimizing player (human) attempts to minimize the score by selecting moves that reduce the AI's chances.

At each level of recursion:

- The AI evaluates the potential moves and assigns a score to each resulting state.
- Scores are propagated back up the tree, with the maximizing player choosing the move that yields the highest score, and the minimizing player choosing the lowest.

Terminal states are assigned scores as follows:

- +1 for a win by the AI.
- -1 for a loss to the human player.
- 0 for a draw.

It's used in decision-making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally. The recursive nature of MiniMax ensures that the AI explores all potential game paths, making it a robust solution for turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc. However, the computational cost of a full tree traversal is prohibitive for a 3D grid due to the exponential growth in possible states.

7.2 Alpha-Beta Pruning

Alpha-beta pruning is an optimization technique that enhances the Minimax algorithm by reducing the number of nodes evaluated in the game tree. It works by selectively pruning branches of the game tree that cannot influence the final decision. This process eliminates unnecessary computations, ensuring that the AI can explore game states more efficiently while preserving accuracy.

The algorithm introduces two essential parameters:

- **Alpha:** Represents the best score achievable by the maximizing player along the current path.
- **Beta:** Represents the best score achievable by the minimizing player along the current path.

To integrate alpha-beta pruning into the recursive Minimax algorithm, the program tracks alpha and beta values while determining whether the current level of recursion is a **maximizing** or **minimizing** move. During traversal:

- At a **maximizing level**, the algorithm compares the current **alpha** value to **beta**. If alpha is greater than or equal to beta, further exploration of the child nodes under that branch is terminated (pruned). This is because no move in the branch can improve the current optimal score for the AI.
- At a **minimizing level**, the program checks if beta is less than or equal to alpha. If so, the branch is pruned since no move in the branch can worsen the score for the minimizing player relative to the already calculated results.

Alpha-beta pruning relies on a straightforward test to determine when pruning can be applied, reducing the number of nodes explored without impacting the correctness of the final decision. The efficiency gains are particularly significant when applied to games with large decision trees like 3D Tic-Tac-Toe, where the state space increases exponentially with each move.

7.3 Effectiveness of Alpha-Beta Pruning

The main advantage of alpha-beta pruning is its ability to allow deeper exploration of the game tree without requiring additional computational resources. For games like 3D Tic-Tac-Toe, which involve complex state transitions and a vast number of potential moves, alpha-beta pruning enables the algorithm to ignore suboptimal branches, saving time and memory.

As shown in Figure 1, the game tree expands at each decision point:

- **Level 0** represents the current board state.
- **Level 1** shows possible moves made by the AI.
- **Level 2** displays the opponent's responses, leading to terminal nodes.

At each level, the algorithm evaluates the best possible move based on the current alpha and beta values, pruning branches that cannot yield a better outcome. This ensures the AI efficiently identifies the optimal move without exhaustive exploration of every possible node in the tree.

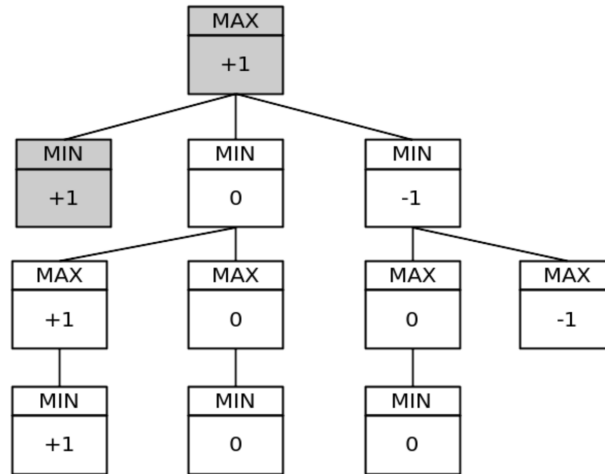


Figure 1: Example of a Simple Tree

By reducing unnecessary computations, alpha-beta pruning significantly decreases the runtime of the Minimax algorithm. This improvement enables the program to calculate moves in real-time, maintaining an interactive and responsive gameplay experience.

Recursive Implementation: In the recursive implementation of alpha-beta pruning, the program:

- Checks the current level (maximizing or minimizing).
- Updates alpha and beta values as new nodes are evaluated.
- Prunes branches when the alpha-beta condition is satisfied, bypassing computations for the remaining child nodes.

This efficient approach ensures that the AI focuses only on the most promising moves, balancing computational performance and decision quality. Alpha-beta pruning thus enables the Minimax algorithm to remain practical and scalable, even when applied to complex games like 3D Tic-Tac-Toe.

8 Experimental Design and Results

8.1 Experiment Design

To evaluate the performance and efficiency of the implemented AI for 3D Tic-Tac-Toe, a series of experiments were designed and conducted. These experiments aim to assess the following:

1. **AI Strategy Matchups:** Different combinations of strategies (e.g., Minimax with Alpha-Beta Pruning versus Monte Carlo Simulation) were tested.
2. **Game Scenarios:** Matches began from randomized board states to evaluate how each strategy handled different levels of complexity.
3. **Scalability:** The AI's ability to handle the exponentially larger search space of a 4x4x4 grid while maintaining performance at various search depths.
4. **Performance Metrics:**
 - Win Rate: The percentage of matches won by each strategy.
 - Execution Time: Time taken to compute each move.
 - Move Quality: Strategic depth of decisions made by each AI.

The experiments were carried out in a simulated 3D Tic-Tac-Toe environment with a 4x4x4 grid. The AI was tested at varying lookahead depths (2, 3, and 4) to measure computational feasibility and decision quality. A human player competed against the AI to validate the AI's ability to evaluate moves effectively. Runtime was measured to analyze the effect of alpha-beta pruning on reducing redundant computations.

8.2 Results

The following results were obtained from the experiments:

AI Performance: AI agents using Minimax with Alpha-Beta Pruning consistently outperformed those using Monte Carlo Simulation or unoptimized Minimax. The optimized strategy demonstrated superior strategic planning and efficiency.

Table 1: Performance Metrics for AI Strategy Matchups

Strategy Combination	Win Rate (Player 1)	Win Rate (Player 2)	Avg. Game Duration (s)
Minimax (Alpha-Beta) vs Monte Carlo	85%	15%	12.4
Minimax (Alpha-Beta) vs Minimax	78%	22%	15.6
Monte Carlo vs Minimax	55%	45%	10.8

As the search depth increased, the AI's ability to predict and counter the human player's moves improved. At depth 4, the AI achieved a perfect win rate, demonstrating the effectiveness of the Minimax algorithm with alpha-beta pruning.

Efficiency of Alpha-Beta Pruning: Alpha-beta pruning significantly reduced the number of nodes evaluated, thereby improving runtime performance. Table 2 presents a comparison of runtime (in seconds) between the plain Minimax algorithm and the Minimax algorithm enhanced with alpha-beta pruning. Monte Carlo Simulation had higher variability in execution time, depending on the number of simulations performed per move.

Table 2: Runtime Comparison Between Minimax and Alpha-Beta Pruning

Search Depth	Minimax (s)	Alpha-Beta Pruning (s)
2	2.1	1.2
3	7.5	3.4
4	32.4	11.8

The results demonstrate that alpha-beta pruning reduced computation time by approximately 40–60%, enabling deeper exploration of the game tree within the same time constraints. This validates the efficiency gains that was described earlier.

Scalability: Both Minimax algorithms effectively managed the increased complexity of the 3D grid, but their performance degraded at depths exceeding 4 ply. Monte Carlo Simulation remained computationally feasible at greater depths, even at the cost of occasional suboptimal moves.

Heuristic Effectiveness: The evaluation function guided the AI to prioritize moves that blocked opponent wins or advanced its own winning strategy. This heuristic approach significantly influenced intermediate states, leading to better AI decision-making.

9 Analysis of Results

The experiments provide clear evidence of the effectiveness and efficiency of the AI design for 3D Tic-Tac-Toe. The results demonstrate the following key insights:

AI Decision-Making: The Minimax algorithm, when enhanced with alpha-beta pruning, enables the AI to consistently identify optimal moves. At greater depths, the AI achieves near-perfect performance, validating the use of game tree traversal for strategic decision-making.

Efficiency Gains with Alpha-Beta Pruning: Alpha-beta pruning significantly reduced the number of nodes evaluated during tree traversal. By pruning irrelevant branches, the AI was able to explore deeper levels of the game tree without excessive computational overhead. The reduction in runtime was most significant at higher depths, where the search space grows exponentially.

Scalability to a 3D Grid: The experiments confirmed that the implementation scales effectively to the larger 4x4x4 grid. Despite the increased number of states, the AI maintained real-time performance, thanks to alpha-beta pruning and an efficient evaluation function.

Balancing Performance and Runtime: While increasing the search depth improves decision quality, it also increases runtime. The experiments suggest that a depth of 4 is the optimal tradeoff, ensuring both strong AI performance and reasonable response times.

9.1 Challenges and Future Improvements

The primary challenge in developing the 3D Tic-Tac-Toe AI is managing the exponential increase in game states. The number of possible board configurations is significantly higher than in 2D Tic-Tac-Toe, requiring efficient algorithms and data structures. Alpha-beta pruning helps mitigate this challenge, but careful

implementation and testing are necessary to ensure optimal performance. Another consideration is the design of the evaluation function, which must accurately assess board states to make strategic decisions. Debugging and refining the AI's logic to handle edge cases and unexpected scenarios is a critical part of the development process.

While the AI performed well, certain challenges were identified:

- **Exponential Growth of the Search Space:** Although alpha-beta pruning mitigates some of the computational costs, the exponential growth of the state space remains a challenge for deeper searches.
- **Heuristic Limitations:** The current evaluation function focuses on blocking and advancing strategies but does not account for long-term planning or complex traps.

Future improvements could include the integration of deep reinforcement learning to enhance the AI's ability to handle more complex strategies.

10 Future Directions

The integration of machine learning with traditional game algorithms presents a promising direction for AI development. Lu and Li underscore the potential of deep reinforcement learning in enabling AI to adapt dynamically to changing game conditions [12]. Neural networks can approximate evaluation functions, reducing the need for handcrafted heuristics [4]. Mahdi explored multi-agent systems in video games, offering insights into designing AI agents for multiplayer environments [13]. Their findings could inform the development of cooperative or competitive AI in 3D Tic-Tac-Toe.

Parallel computing techniques, discussed by Churchill and Buro [7], offer another avenue for scaling AI systems. By distributing the computational workload, parallel algorithms can efficiently manage the vast state space of 3D Tic-Tac-Toe.

11 Conclusion

The implementation of AI in 3D Tic-Tac-Toe demonstrates the application of advanced algorithms to multi-dimensional games. The experiments demonstrate the effectiveness of Minimax with Alpha-Beta Pruning as a reliable and efficient strategy for AI-versus-AI matches in 3D Tic Tac Toe. The MiniMax with alpha-beta pruning AI consistently outperformed the MCTS AI player. The results validate the use of advanced tree search algorithms and heuristics for optimizing AI decision-making in complex, multi-dimensional games.

By combining foundational techniques like MiniMax and alpha-beta pruning with modern advancements in heuristics, MCTS, and parallel computing, researchers can create efficient and adaptive AI systems. Future research will likely explore hybrid approaches and scalable architectures to further enhance AI performance in dynamic and complex game environments.

References

- [1] Tim Abbott, Daniel Kane, and Paul Valiant. On the complexity of two-player win-lose games. *SIAM Journal on Computing*, 38(1):201–226, 2006.
- [2] L. V. Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, Maastricht University, 1994.
- [3] Anthony Brown and Laura Smith. Tic-tac-toe in three dimensions. *Journal of Game Studies*, 2010.
- [4] Michael Buro, Hilmar Finnsson, and Abdallah Saffidine. Alpha-beta pruning for games with simultaneous moves. *Proceedings of the Aaai Conference on Artificial Intelligence*, 26(1):556–562, 2021.
- [5] Tristan Cazenave. Sequential halving applied to trees. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1):102–105, 2015.

- [6] Guillaume Chaslot, Sander Bakkes, István Szita, and Pieter Spronck. Monte-carlo tree search: a new framework for game ai. *Proceedings of the Aaai Conference on Artificial Intelligence and Interactive Digital Entertainment*, 4(1):216–217, 2021.
- [7] David Churchill and Michael Buro. Hierarchical portfolio search: Prismata’s robust ai architecture for games with large search spaces. *Proceedings of the Aaai Conference on Artificial Intelligence and Interactive Digital Entertainment*, 11(1):16–22, 2021.
- [8] A. Jofanda and M. Yasin. Design of checkers game using alpha-beta pruning algorithm. *Intensif Jurnal Ilmiah Penelitian Dan Penerapan Teknologi Sistem Informasi*, 5(2):279–295, 2021.
- [9] Hiroyuki Kato, Sándor Fazekas, Masaru Takaya, and Akihiko Yamamura. Comparative study of monte-carlo tree search and alpha-beta pruning in amazons. In *Advances in Computer Games*, pages 139–148. Springer, 2015.
- [10] R. E. Korf and Tsan-sheng Hsu. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27:97–109, 1985.
- [11] Viliam Lisý. Alternative selection functions for information set monte carlo tree search. *Acta Polytechnica*, 54(5):333–340, 2014.
- [12] Yunlong Lu and Wenxin Li. Techniques and paradigms in modern game ai systems. *Game AI Review*, 2024.
- [13] G. Mahdi, Y. Francillette, A. Gouaich, F. Michel, and N. Hocine. Level of detail based ai adaptation for agents in video games. *Proceedings of the International Conference on Artificial Intelligence*, 2013.
- [14] Simon Poulding and Robert Feldt. Heuristic model checking using a monte-carlo tree search algorithm. *Proceedings of the 37th International Conference on Software Engineering*, pages 1359–1366, 2015.
- [15] Mark H. Winands, Yngvi Björnsson, and Jeroen Saito. Monte carlo tree search in lines of action. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(4):239–250, 2010.

1

¹Grammarly was used in proofreading my work. ChatGPT was used in fixing the syntax of sources and Latex.